



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Object Class Feature

Pemrograman Berorientasi Objek

Compare (ver 1)

```
public static void compare1() {  
    Object obj1 = new Object();  
    Object obj2 = new Object();  
    Object obj3 = obj1;  
  
    System.out.println(obj1==obj2);  
    System.out.println(obj1==obj3);  
    System.out.println(obj1.equals(obj2));  
    System.out.println(obj1.equals(obj3));  
    System.out.println(obj1.hashCode());  
    System.out.println(obj2.hashCode());  
    System.out.println(obj3.hashCode());  
}
```

Output (ver 1)

false

true

false

true

746292446

1072591677

746292446

Compare (ver 2)

```
public static void compare2() {  
    String st1 = new String("Halo");  
    String st2 = new String("Halo");  
    String st3 = st1;  
  
    System.out.println(st1==st2);  
    System.out.println(st1==st3);  
    System.out.println(st1.equals(st2));  
    System.out.println(st1.equals(st3));  
    System.out.println(st1.hashCode());  
    System.out.println(st2.hashCode());  
    System.out.println(st3.hashCode());  
}
```

Output (ver 2)

false

true

true

true

2241628

2241628

2241628

Compare (ver 3)

```
public static void compare3() throws CloneNotSupportedException {  
    Point p1 = new Point(5,10);  
    Point p2 = new Point(5,10);  
    Point p3 = p1;  
  
    System.out.println(p1==p2);  
    System.out.println(p1==p3);  
    System.out.println(p1.equals(p2));  
    System.out.println(p1.equals(p3));  
    System.out.println(p1.hashCode());  
    System.out.println(p2.hashCode());  
    System.out.println(p3.hashCode());  
  
    p3 = (Point) p1.clone();  
    System.out.println(p1);  
    System.out.println(p3);  
    System.out.println(p1==p3);  
    System.out.println(p1.equals(p3));  
    System.out.println(p3.hashCode());  
}
```

Class Point

```
public class Point implements Cloneable {  
    private int x = 0;  
    private int y = 0;  
    // a constructor!  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public int getX() {  
        return x; }  
    public int getY() {  
        return y; }  
    public void setX(int ax) {  
        x = ax; }  
    public void setY(int ay) {  
        y = ay; }  
  
    @Override  
    public String toString() {  
        return "Point [x=" + x + ", y=" + y + "]";  
    }  
}
```

Class Point (...)

```
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Point other = (Point) obj;
    if (x != other.x)
        return false;
    if (y != other.y)
        return false;
    return true;
}
```


Class Point (...)

```
@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + x;
    result = prime * result + y;
    return result;
}

@Override
protected Object clone() throws CloneNotSupportedException {
    return super.clone();
}
}
```

Output (ver 3)

false

true

true

true

1126

1126

1126

Point [x=5, y=10]

Point [x=5, y=10]

false

true

1126

Compare (ver 4)

```
public static void compare4() throws CloneNotSupportedException {  
    Point p = new Point(10,20);  
    Rectangle r1 = new Rectangle(p,30,40);  
  
    System.out.println(p);  
    System.out.println(r1);  
  
    Rectangle r2 = (Rectangle) r1.clone();  
    System.out.println(r2);  
    System.out.println(r1==r2);  
    System.out.println(r1.equals(r2));  
    System.out.println(r1.getOrigin()==r2.getOrigin());  
    System.out.println(r1.hashCode());  
    System.out.println(r2.hashCode());  
}
```

Class Rectangle

```
public class Rectangle implements Cloneable {  
    public int width = 0;  
    public int height = 0;  
    public Point origin;  
    // four constructors  
    public Rectangle() {  
        origin = new Point(0, 0); }  
    public Rectangle(Point p) {  
        origin = p; }  
    public Rectangle(int w, int h) {  
        origin = new Point(0, 0);  
        width = w; height = h; }  
    public Rectangle(Point p, int w, int h) {  
        origin = p; width = w; height = h; }  
    // a method for moving the rectangle  
    public void move(int x, int y) {  
        origin.setX(x);  
        origin.setY(y);  
    }  
}
```

Class Rectangle (...)

```
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Rectangle other = (Rectangle) obj;
    if (height != other.height)
        return false;
    if (width != other.width)
        return false;
    if (origin == null) {
        if (other.origin != null)
            return false;
    } else if (!origin.equals(other.origin))
        return false;
    return true;
}
```

Class Rectangle (...)

```
@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + height;
    result = prime * result + width;
    result = prime * result + ((origin == null) ? 0 :
origin.hashCode());
    return result;
}

// @Override
// protected Object clone() throws CloneNotSupportedException {
//     return super.clone();
// }

@Override
protected Object clone() throws CloneNotSupportedException {
    Rectangle tmp = (Rectangle) super.clone();
    tmp.origin = (Point) origin.clone();
    return tmp;
}
```

Output (ver 4)

Point [x=10, y=20]

Rectangle [width=30, height=40, origin=Point [x=10, y=20]]

Rectangle [width=30, height=40, origin=Point [x=10, y=20]]

false

true

false

70452

70452

Terima Kasih